Capstone Report – Mini-Courier Planner

1. Problem Statement

The project builds an intelligent courier-planning prototype. Given a list of parcels, their delivery points, deadlines, priorities, weights and values, the system selects a valuable feasible set of parcels and generates an ordered delivery route. The solution integrates sorting, dynamic programming, graph algorithms and approximation techniques.

2. Objectives

• Prioritize parcels according to delivery deadline and priority.

• Select an optimal subset of parcels subject to truck capacity.

• Find shortest paths between delivery points.

• Construct an overall delivery route using a TSP approximation.

• Measure runtime and discuss scalability.

• Provide a modular, executable Python implementation.

3. System Design

Parcel CSV + Weighted Graph CSV
|
v
Parcel Prioritization
(Merge Sort)
|
v
Box Filling
(0/1 Knapsack DP)
|
v
Selected Delivery
Locations
|
v
Shortest Paths
(Dijkstra)
|
v
Overall Route Planning
(Nearest-Neighbor TSP)
|
v
Final Route + Metrics
|

v

Runtime / Evaluation

4. Dataset Design

Parcel attributes

• `parcel_id`: unique parcel identifier.

• `item`: parcel category.

• `destination`: delivery location.

• `deadline`: required delivery date.

• `priority`: Critical/High/Medium/Low.

• `weight`: parcel weight in kg.

• `value`: business value used by the packing objective.

• `volume`: package volume.

Graph attributes

• `source`

• `destination`

• `distance_km`

The sample graph represents a small Delhi-NCR-style courier network with a warehouse and five delivery locations.

5. Phase 1 – Courier Problem Design and Planning

The problem is divided into four algorithmic stages:

1. Sort all parcels so urgent shipments are considered first.

2. Use 0/1 Knapsack DP to maximize total parcel value without exceeding truck capacity.

3. Use Dijkstra to calculate shortest paths in the weighted graph.

4. Use nearest-neighbor TSP approximation to visit all selected destinations and return to the warehouse.

This modular design allows each algorithm to solve the part of the problem for which it is suitable.

6. Phase 2 – Package Prioritization and Box Filling

Sorting

Merge Sort is used because it guarantees O(n log n) time and is stable. The sorting key is:

1. Earlier deadline.

2. Higher priority.

3. Lower weight as a final tie-breaker.

Dynamic Programming

The packing problem is modeled as 0/1 Knapsack:

• Weight = parcel weight.

• Value = parcel value.

• Capacity = 25 kg.

• Each parcel can be selected at most once.

The DP state is:

`dp[i][c] = maximum value using the first i parcels with capacity c.`

Transition:

`dp[i][c] = max(dp[i-1][c], dp[i-1][c-w_i] + v_i)` when the parcel fits.

The final selected list is reconstructed by tracing the DP table backwards.

7. Phase 3 – Route Maker and Graph Processing

Dijkstra

The weighted graph uses non-negative distances. Dijkstra's algorithm computes shortest distances efficiently using a priority queue.

Nearest-Neighbor TSP Approximation

Exact TSP becomes expensive as the number of stops increases. Therefore, starting at the warehouse, the system repeatedly visits the nearest unvisited destination according to shortest-path distance, then returns to the warehouse.

This is a heuristic/approximation, so it is fast but does not guarantee the globally optimal TSP tour.

8. Phase 4 – Performance Testing

The benchmark tests input sizes 10, 50, 100, 200 and 500.

Complexity expectations

• Merge Sort: O(n log n).

• Knapsack DP: O(nW), where W is capacity.

• Dijkstra with a binary heap: O((V+E) log V).

• Nearest-neighbor selection: $O(k^2)$ after pairwise shortest-path distances are available.

The benchmark CSV and charts are included in `outputs/` and `images/`.

Observation: sorting is expected to scale gently with n. Knapsack is strongly influenced by both the number of parcels and the capacity W. Dijkstra depends on graph size and edge density. Nearest-neighbor is substantially cheaper than exact TSP and is therefore appropriate for a prototype.

9. Results

The application reports:

• prioritized parcel order,

• selected parcel IDs,

• total selected weight,

• total selected value,

• an example shortest path,

• final approximate route,

• route distance,

• truck utilization.

The exact generated values are recorded by the executable program and can be regenerated with the commands in the README.

10. Correctness Checks

• The packing solution never exceeds the configured truck capacity.

• Each parcel is either selected or not selected, satisfying the 0/1 constraint.

• Dijkstra returns minimum known distances for the non-negative weighted graph.

• The route starts and ends at the warehouse.

• Every selected destination is visited once in the approximate route.

• Benchmark results are written to CSV for reproducibility.

11. Optimization / Future Scope

1. Replace nearest-neighbor with 2-opt or a stronger TSP heuristic.

2. Use time windows for delivery deadlines.

3. Add multiple vehicles and vehicle capacities.

4. Include traffic-dependent travel time.

5. Use priority-aware multi-objective optimization instead of value-only knapsack.

6. Add a map API and real geographic coordinates.

7. Use a faster graph representation for very large datasets.

8. Add a graphical dashboard for interactive route planning.

12. Conclusion

Mini-Courier Planner demonstrates how multiple algorithmic paradigms can be integrated into one practical application. Sorting handles prioritization, dynamic programming handles constrained package selection, Dijkstra handles shortest paths, and nearest-neighbor provides a scalable route approximation. The modular implementation, datasets, runtime measurements and documentation satisfy the major deliverables of the capstone assignment.

13. Submission Checklist

✓ README.md

✓ mini_courier_planner.ipynb

✓ src/ modular source code

✓ datasets/

✓ images/

✓ outputs/

✓ requirements.txt

✓ .gitignore

✓ report

■ Push to private GitHub repository

■ Tag final commit as `v1.0-final-submission`

■ Submit repository URL through LMS/Zosima